

Sunnyside Labs PrivacyBoost Audit



March 19, 2026

Table of Contents

Table of Contents	2
Summary	4
Scope	5
System Overview	6
Key Components	6
Transaction Flow	7
Security Model and Trust Assumptions	7
Privacy Model	7
Trust Assumptions	8
Privileged Roles	8
Censorship Resistance	8
Critical Severity	9
C-01 Tree Number Range Misconstraint Causes Permanent Protocol Halt After 16 Rollovers	9
Medium Severity	10
M-01 Low-Order BabyJubJub Keys Enable EdDSA Forgery	10
M-02 accountId Squatting via First-Come Ownership Assignment	10
M-03 Verifier Upgrade Can Break Forced Withdrawal Escape Hatch	11
M-04 Forced Withdrawal Can Be Blocked by Spending Requested Nullifiers During Delay	12
M-05 Forced Withdrawal Execution is Requester-Only, Enabling DoS by Front-Running	12
M-06 Lazy Auth Snapshot Allows Revoked Keys to Remain Valid for Current Round	13
Low Severity	14
L-01 Incorrect Subgroup Order Constant in EdDSA Canonicity Check	14
L-02 Implicit Ascending Order Assumption in Withdrawal Slots	14
L-03 Inconsistent Zero-Check in Constructor	15
L-04 Withdrawal Slots Validated After Use	15
L-05 Floating and Outdated Pragma	15
L-06 Possible Violation of 'No Conflicting Names' EEA EthTrust Security Level [S] Requirement	16
L-07 Risk of De-Anonymization if Total Transfers or Outputs per Deposit Epoch Are Too Few	16
L-08 Non-Canonical Encoding of digestRootIndices	17
L-09 Mutable authSnapshotInterval Can Skew currentRound and Break Proof Validity	18
L-10 Forced-Withdrawal Cancellation and Execution Delays Are Equal, Making Cancellation a Race	18
L-11 Forced Withdrawal Requests May Lock Commitments if Execution Reverts	19
L-12 revoke Typed Data Lacks Expiry, Allowing Delayed Relay Execution	19
L-13 Non-Canonical Commitments Accepted in Deposit Requests	20

Notes & Additional Information	21
N-01 Missing Public Input Length Check in Base Verifier	21
N-02 Unnecessary unchecked Blocks	21
N-03 Redundant Checks, Getters, Caching, and Returns	22
N-04 Constraint Optimizations	22
N-05 Misleading Comments	23
N-06 Gas Optimizations	24
N-07 Unused usedRoots Entries Permit Non-Canonical Public Inputs	24
N-08 OutputNPK–Ciphertext Mismatch Can Produce Unspendable Notes	25
N-09 VK Registry Key Does Not Include Full Circuit Shape	25
N-10 Possible Redundant (treeNumber, root) Entries In Sparse Roots List	26
N-11 Missing and Incomplete Docstrings	26
N-12 Deposit Circuit Permits Zero-Value Output Commitments	27
N-13 MSM Failure Reverts With Misleading Error	28
Conclusion	29
Appendix	30
Issue Classification	30

Summary

Type	Privacy	Total Issues	33 (27 resolved, 1 partially resolved)
Timeline	From 2026-02-11 To 2026-03-04	Critical Severity Issues	1 (1 resolved)
Languages	Solidity Go	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	6 (4 resolved, 1 partially resolved)
		Low Severity Issues	13 (11 resolved)
		Notes & Additional Information	13 (11 resolved)

Scope

OpenZeppelin performed a security audit of the [testinprod-io/privacy-boost-protocol](https://github.com/testinprod-io/privacy-boost-protocol) repository at commit [7b2a018](#).

In scope were the following files:

```
contracts/src
├─ PrivacyBoost.sol
├─ AuthRegistry.sol
├─ TokenRegistry.sol
├─ hash/
│   └─ Poseidon2T4.sol
├─ interfaces/
│   ├── Constants.sol
│   └─ IStructs.sol
├─ lib/
│   ├── LibAuthZeroHashes.sol
│   ├── LibBabyJubJub.sol
│   ├── LibDigest.sol
│   ├── LibMerkleTree.sol
│   ├── LibPublicInputs.sol
│   └─ LibZeroHashes.sol
└─ verifier/
    ├── Groth16Verifier.sol
    ├── Groth16DepositVerifier.sol
    ├── Groth16EpochVerifier.sol
    └─ Groth16ForcedVerifier.sol

frontend/
├─ auth_merkle.go
├─ bool.go
├─ constants.go
├─ deposit_epoch_circuit.go
├─ eddsa_poseidon_verifier.go
├─ epoch_circuit.go
├─ forced_withdraw_circuit.go
├─ helpers.go
├─ merkle_fixed.go
└─ poseidon2t4.go
```

System Overview

PrivacyBoost is an epoch-based shielded pool that enables confidential token transfers on Ethereum or any EVM-compatible chain. Users deposit tokens into the pool and receive private notes (UTXO-style commitments). These notes can be transferred privately or withdrawn, which is achieved by spending/nullifying an input note and, in the transfer case, generating one or more output notes. The details of each transaction are encrypted using a 3-way ECDH encryption scheme. The ciphertext is posted as calldata on-chain, and may only be decrypted and viewed by a Trusted Execution Environment (TEE) and the recipient. However, only the user can authorize new transfers or withdrawals as this requires a signature with the user's secret EdDSA authorization key.

The protocol uses a UTXO model where notes are commitments of the form `Poseidon(npk, tokenId, value, rnd)`. When spending, users reveal a nullifier derived from their secret key and the commitment, which prevents double-spending without revealing which note was spent. All notes are stored in append-only Merkle trees (depth 20, up to 2^{20} leaves per tree). Leaves are only appended to one tree (known as the active tree) at a time. When this tree reaches its full capacity, a "rollover" is initiated, turning the previous active tree into a fixed historical tree and naming a new, empty tree as the active tree.

Transactions are batched into "epochs" by a relay operating within a TEE. The TEE relayer aggregates user requests and generates a single Groth16 proof covering the entire batch, which is verified on-chain. This batching improves gas efficiency and enlarges the anonymity set per submission. Zero-knowledge proofs validate that each transaction is authorized (i.e., correctly EdDSA-signed by a user whose EdDSA public key is in the AuthKey registry), that the notes are spendable by that user, that sum of the output values plus applicable fees is equal to the sum of the input values, and that the Merkle root of the note tree is updated correctly after each epoch.

Key Components

- `PrivacyBoost.sol` is the core pool contract that handles deposits, epoch submissions, and withdrawals.
- `AuthRegistry.sol` manages user authentication keys (BabyJubJub public keys) with round-based snapshots.

- **Groth16 Verifiers** perform the on-chain verification of ZK proofs for epochs, deposits, and forced withdrawals.
- **ZK Circuits (gnark)** prove transaction validity, including note ownership, Merkle membership, and signature verification.

Transaction Flow

1. **Deposit:** User calls `requestDeposit()`, creating pending commitments. The relay batches and submits via `submitDepositEpoch()`.
2. **Transfer:** User signs a transfer request. The relay batches multiple transfers and submits via `submitEpoch()`.
3. **Withdrawal:** Included in epoch submission, revealing recipient and amount but not the source note.
4. **Forced Withdrawal:** Censorship-resistance mechanism allowing users to exit without relay cooperation after a delay period.

Security Model and Trust Assumptions

Privacy Model

Action	Public	Hidden
Deposit	depositor address, token ID, total amount, output commitments, ciphertext blobs	per-output amounts/recipients/note randomness (inside encrypted metadata + witness)
Transfer	input nullifiers, output commitments, zk proof, selected roots/tree IDs, fee commitments	input-output linkage, per-note amounts, ownership secrets
Withdrawal	withdrawal destination address, token ID, amount, nullifiers spent in that tx	input-output linkage and per-output amounts (beyond public nullifiers)

Privacy is achieved on the spend side: observers cannot link nullifiers to their source commitments. During private transfers, the sender and recipient remain private at the protocol level, but encrypted metadata may be decrypted by authorized third parties such as the TEE.

Trust Assumptions

Throughout the in-scope codebase, the following trust assumptions were identified:

- **TEE/Relay:** Trusted for liveness and privacy (sees plaintext transaction data), but not for fund custody. Users can bypass via forced withdrawal.
- **ZK Soundness:** Groth16 proofs guarantee that invalid state transitions cannot be accepted.
- **Trusted Setup:** The protocol relies on a properly executed Groth16 trusted setup ceremony for soundness.
- **Off-chain Robustness:** Off-chain workers (TEE, indexers) are assumed to implement standard robustness controls (bounded retries, dead-letter handling, rate limiting) to gracefully handle malformed or inconsistent ciphertext.

Privileged Roles

Throughout the in-scope codebase, the following privileged roles and actions were identified:

Role	Capabilities
Owner	Set verifiers, treasury, fees. Can upgrade verifiers immediately (no timelock).
Operator	Manage allowed relays, set auth snapshot interval, update transaction public key.
Relay	Submit epochs. Cannot forge proofs or steal funds, but can censor transactions.

Censorship Resistance

The forced withdrawal mechanism provides an escape hatch: users can submit their own ZK proof and withdraw after a delay period, bypassing relay censorship. This guarantee depends on the ZK circuits correctly validating forced withdrawal proofs.

Critical Severity

C-01 Tree Number Range Misconstraint Causes Permanent Protocol Halt After 16 Rollovers

Tree number values are constrained in all three circuits (listed below) to be `< Shape.MaxTrees` (typically 16), even though the contract defines tree numbers as a monotonically increasing 15-bit identifier up to 32,768:

- `epoch_circuit.go` at (1, 2, 3)
- `deposit_epoch_circuit.go`
- `forced_withdraw_circuit.go`

The circuit shape parameters represent sparse array size (how many tree roots are included per proof), not the allowed value domain of tree identifiers. By range-checking tree numbers against 16, the circuits incorrectly treat the global tree number as a small index rather than a protocol-wide identifier.

Once the protocol rolls over the note or auth tree 16 times and `currentTreeNumber` becomes 16, the contract requires `activeTreeNumber == currentTreeNumber`, while the circuit enforces `ActiveNoteTreeNumber < 16`. At that point, no valid proof can be constructed, causing deposits, transfers, withdrawals, and forced withdrawals to halt permanently. As a result, all the user funds become inaccessible, and the forced withdrawal escape hatch for the protocol's censorship resistance mechanism is also broken. Since the contract enforces equality with the global tree number, there is no workaround, and all protocol operations become unsatisfiable until the circuits are replaced.

In addition, the constants file (`constants.go`) defines `NoteTreeNumberBits = 5` and `AuthTreeNumberBits = 5` for range checks, while `TreeNumberBitsPerSlot = 15` is used for packing. This inconsistency means that even if the `< Shape.MaxTrees` bound were corrected, the 5-bit decomposition would still fail for tree numbers greater than or equal to 32. As such, both `NoteTreeNumberBits` and `AuthTreeNumberBits` should be updated to 15 to match the protocol's 15-bit tree number domain.

Consider removing the redundant `< Shape.MaxTrees` range checks entirely, as tree validity is already enforced through (`treeNumber`, `root`) matching logic. Alternatively, constrain tree numbers against the correct 15-bit domain (e.g., `< 32768`) instead of the sparse array size.

Either approach requires circuit recompilation and a new trusted setup to restore long-term protocol viability.

Update: Resolved in [pull request #164](#).

Medium Severity

M-01 Low-Order BabyJubJub Keys Enable EdDSA Forgery

The gnark EdDSA verifier `computeEdDSAVerifyArtifacts` performs cofactor clearing on the public key, computing $A8 = 8 * A$, and confirms that A and $R8$ both lie on the curve via `isOnCurve`. However, neither the verifier nor `isOnCurve` enforce that the points lie in the prime-order subgroup. As a result, low-order torsion points (including the identity $(0, 1)$) can be used as public keys.

If a low-order public key is registered, cofactor clearing maps it to the identity ($A8 = 0$). The signature equation then becomes message-independent: $S * B8 = R8 + h * A8 = R8$ when $A8 = 0$. This removes the message-binding term and allows signatures to be constructed without knowledge of a corresponding secret key, breaking the intended authentication guarantee for such keys.

Consider enforcing a check during key registration/rotation to reject low-order torsion public keys by rejecting any public key whose cleared form equals the identity (i.e., reject $A8 = (0, 1)$).

Update: Resolved in [pull request #238](#) at commits [784a19b](#), [a79ed6e](#), and [6643a5b](#).

M-02 `accountId` Squatting via First-Come Ownership Assignment

In `AuthRegistry.sol`, the `register` function assigns `account.owner = expectedOwner` upon first registration, without any cryptographic binding between `accountId` and the owner. The EIP-712 signature proves control of `expectedOwner`, but does not prove entitlement to the chosen `accountId`. Thus, ownership is effectively first-come-first-served.

An attacker can claim any unused `accountId` by submitting a valid self-signed registration. Subsequent registrations for the same `accountId` revert with `OwnerMismatch()`, permanently blocking the intended user. In public mempools, pending registrations can be front-run. Predictable `accountId`s can be proactively pre-claimed regardless. If notes reference an `accountId` before ownership is established, those funds become unspendable since spend and forced-withdrawal proofs require auth-tree membership for that `accountId`.

Consider binding `accountId` to `expectedOwner` on-chain (e.g., `accountId = hash(owner, salt)` validated during registration), or introducing an ownership transfer/recovery mechanism. If first-come-first-served is intended by design, document this assumption and require accounts to register before receiving value.

Update: Resolved in [pull request #239](#) at commit [3db0d9e](#).

M-03 Verifier Upgrade Can Break Forced Withdrawal Escape Hatch

The owner can update verifier contracts (`epochVerifier`, `depositVerifier`, and `forcedVerifier`) at any time without a timelock or delay. This includes `forcedVerifier`, which is responsible for validating proofs used in the forced withdrawal flow. Since updates take effect immediately, users must trust the owner not to change verification rules arbitrarily.

Forced withdrawal exists to provide a censorship-resistant exit in case the relay refuses to include user transactions. However, owner control over `forcedVerifier` means that this exit path can be disabled or altered at any time. A malicious or compromised owner could deploy a verifier that rejects all proofs, preventing users from completing forced withdrawals and trapping funds.

Consider introducing a timelock (e.g., 24–48 hours) for verifier updates so that users can react before changes take effect. Alternatively, make `forcedVerifier` immutable after deployment or restrict upgrades through a governance mechanism to ensure that the forced withdrawal escape hatch cannot be disabled unexpectedly.

Update: Resolved in [pull request #240](#) at commit [822501b](#).

M-04 Forced Withdrawal Can Be Blocked by Spending Requested Nullifiers During Delay

In `PrivacyBoost.sol`, the `requestForcedWithdrawal` function verifies that the provided nullifiers are unspent, but does not reserve them during the forced withdrawal delay. A relay holding previously signed user transactions spending the same nullifiers can include them during the delay period. As a result, the `executeForcedWithdrawal` function reverts because the nullifiers are already spent.

This allows the relay to block the forced withdrawal escape hatch in the adversarial scenario it is intended to address. A user may request forced withdrawal after observing non-inclusion of their transactions, but the relay can include the previously withheld signed transactions before the withdrawal executes, thus blocking forced withdrawal execution. This undermines the censorship resistance of the protocol and contradicts the stated goal that forced withdrawal provides a self-custody exit without trusting any TEE or off-chain infrastructure.

Consider reserving nullifiers at request time, marking them as pending until execution or expiry. Alternatively, consider introducing a user-controlled nonce mechanism that allows users to invalidate previously signed relay transactions before initiating forced withdrawal.

Update: Acknowledged, not resolved. The Sunnyside Labs team stated:

We think that both of the suggested solutions do not completely solve the issue but reduce attack surface. Nullifier reservation: the attacker can still front-run within the next block. Nonce mechanism: similar could happen due to lazy auth snapshots.

M-05 Forced Withdrawal Execution is Requester-Only, Enabling DoS by Front-Running

When `executeForcedWithdrawal` is called, it calls `_validateForcedWithdrawalRequest` with `allowOwnerCancel == false`. This forces `msg.sender` to match the stored `requester`, or the forced withdrawal execution `reverts`. If, when a user calls `requestForcedWithdrawal`, an attacker front-runs their transaction and copies their proof, the attacker becomes the stored `requester`. Subsequent attempts by the user to execute the forced withdrawal after the delay period will revert and, if the attacker refuses to execute the forced withdrawal, the user can only cancel their forced withdrawal request and re-attempt it. A persistent attacker could lock their funds indefinitely. The forced withdrawal proof is bound to `withdrawal.to`, so the recipient of the funds cannot be changed.

Consider making `executeForcedWithdrawal` permissionless, or at least allowing `withdrawalTo`/account owner execution, to prevent a front-running attacker from blocking a user's forced withdrawal.

Update: Resolved in [pull request #241](#) at commits [709d059](#) and [8d118b4](#).

M-06 Lazy Auth Snapshot Allows Revoked Keys to Remain Valid for Current Round

Authorization roots are snapshotted lazily per round via `__snapshotAuthTreeIfNeeded`. When a snapshot for `(round, treeNum)` is first accessed, the contract records the current root from `AuthRegistry.authTreeRoot` and stores it in `authSnapshots`. The snapshot is written only once and is not refreshed again during the same round.

Since `AuthRegistry` updates auth-tree roots immediately on `rotate` and `revoke`, a transaction that triggers the snapshot early in a round can lock in a pre-rotation root. Since functions such as `requestForcedWithdrawal` are permissionless, an attacker holding a soon-to-be-revoked key can trigger the snapshot before revocation occurs. As a result, proofs using the revoked key may continue to validate against the snapshotted root until the round changes (`authSnapshotInterval`). This allows revoked or rotated keys to remain usable for the remainder of the round, weakening revocation guarantees.

Consider snapshotting authorization roots deterministically at round boundaries (e.g., snapshot once per round at a predetermined point, such as the first epoch submission). This removes the first-caller-wins race while preserving proof stability. Refreshing snapshots on root changes would fix the race but can invalidate in-flight proofs.

Update: Partially Resolved in [pull request #297](#) at commit [8e43004](#). The Sunnyside Labs team has stated that snapshotting is now restricted to the relay as a best-effort mitigation, and that deterministic snapshotting would not fully resolve the issue.

Low Severity

L-01 Incorrect Subgroup Order Constant in EdDSA Canonicity Check

The implementation enforces `S < subgroupOrder` using a hardcoded BabyJubJub subgroup order constant, but the stored value is `l - 1` instead of the true order `l`. This results in enforcing `S < l - 1` rather than `S < l`, causing the verifier to reject the single valid scalar value `S = l - 1`. The impact is limited to rejecting one valid canonical signature, creating a minimal denial-of-service edge case with negligible probability (`1 / l`), but deviating from the EdDSA specification and potentially affecting interoperability.

Consider updating the hardcoded subgroup order constant to the exact BabyJubJub subgroup order value

```
(2736030358979909402780800718157159386076813972158567259200215660948447373041)
```

to restore the correct canonical bound without modifying the surrounding verification logic.

Update: Resolved in [pull request #242](#) at commit [25c6e85](#).

L-02 Implicit Ascending Order Assumption in Withdrawal Slots

The `_computeTransferDigests` function uses a forward-only cursor and only matches withdrawals when `withdrawalSlots[withdrawalCursor] == uint32(t)` during an increasing loop. This implicitly assumes `withdrawalSlots` are strictly ascending and unique, but that invariant is not enforced. If slots are unsorted or duplicated, later entries can become unreachable, and the function reverts only at the end via a cursor mismatch. The behavior depends on ordering rather than explicit validation.

Consider enforcing strict ascending and unique ordering of `withdrawalSlots` during input validation with a dedicated revert reason to make the invariant explicit and avoid wasted computation.

Update: Resolved in [pull request #243](#) at commit [0b05483](#).

L-03 Inconsistent Zero-Check in Constructor

The constructor for the `PrivacyBoost` contract `checks` that `maxForcedInputs_` is non-zero, but does not perform similar validation for other constructor parameters. This results in inconsistent input validation during deployment. If defensive validation is intended, parameters that must satisfy protocol invariants (e.g., non-zero or bounded values) should be validated consistently. Otherwise, the isolated check introduces an unnecessary inconsistency.

Consider validating all constructor parameters that must satisfy protocol invariants, or removing the isolated `maxForcedInputs_` zero check for consistency.

Update: Resolved in [pull request #244](#) at commits [e36a352](#) and [7a24a4a](#).

L-04 Withdrawal Slots Validated After Use

In `PrivacyBoost.sol`, the `submitEpoch` function validates `withdrawalSlots.length == withdrawals.length` and slot bounds only after `_computeTransferDigests` has already consumed `withdrawalSlots`. This allows malformed input to reach expensive digest computation before being rejected and wastes gas. It may also result in generic panics rather than explicit custom errors as invalid batches are partially processed before failing.

Consider moving all `withdrawalSlots` length and bounds checks to the top-level input validation in `submitEpoch`, before `_computeTransferDigests` is called, so that malformed input fails fast.

Update: Resolved in [pull request #243](#) at commit [0b05483](#).

L-05 Floating and Outdated Pragma

Pragma directives should be fixed to clearly identify the Solidity version with which the contracts will be compiled. In addition, it is best practice to use the latest version of Solidity when possible.

The `Poseidon2T4.sol` file has the `^0.8.21` floating pragma directive. In addition, the `AuthRegistry.sol`, `PrivacyBoost.sol`, and `TokenRegistry.sol` files use Solidity version `0.8.28`.

Consider using fixed pragma directives. Moreover, consider using the latest stable version of Solidity, which is 0.8.34 at the time of writing.

Update: Resolved in [pull request #245](#) at commit [b7208af](#).

L-06 Possible Violation of 'No Conflicting Names' EEA EthTrust Security Level [S] Requirement

Possible violation of the [EEA EthTrust Security Level \[S\]](#) “No Conflicting Names” requirement. This requirement states that function and variable names should not be repeated in a way that could introduce confusion. Function overriding from a base contract is permitted, provided only one effective implementation is used within the code.

Within the `LibDigest.sol` file, there is a possible conflict:

- The `computeCommitmentsHash` function
- The `computeCommitmentsHash` function

Consider addressing the conflicting names.

Update: Resolved in [pull request #251](#) at commit [01e5447](#).

L-07 Risk of De-Anonymization if Total Transfers or Outputs per Deposit Epoch Are Too Few

The number of transfers (`nTransfers`) in an [epoch](#) and the total number of output commitments (`nTotalCommitments`) in a [deposit epoch](#) is publicly visible, and only required by the protocol and constrained by the circuits to be at least 1. If the total number of transfers or the number of transfers in a deposit epoch is low, an observer may be able to correlate notes with deposits, de-anonymizing the note and revealing its value.

For epochs, processing shielded-to-shielded transfers and withdrawals, all notes in the tree contribute to the anonymity set. As long as there have been enough commitments appended to the note tree since the beginning of the protocol, output note values are sufficiently obfuscated. By the same logic, for epochs, as long as the number of depositors since the beginning of the protocol is large enough, note ownership remains obfuscated, preserving anonymity. Loss of privacy may occur beginning at the protocol's launch and continuing until many deposits and transactions have been processed. This can be avoided by bootstrapping the system, having the TEE generate dummy transactions until a sufficient number of users have joined the protocol.

However, since deposit requests are public, the anonymity set of each *deposit* epoch is smaller, consisting only of depositors within that deposit epoch (or a deposit epoch in the near future, if the TEE must delay processing of their deposit for whatever reason). If few output notes are generated in a single deposit epoch (in the worst case, one deposit and one output note), an output note may be correlated with a deposit request.

To prevent loss of privacy in epochs, consider bootstrapping the system by generating dummy output notes until user-generated traffic is sufficient to make de-anonymization of notes infeasible. To prevent loss of privacy in deposit epochs, consider using fixed-sized batches, padded with dummy (zero-valued) transactions if the anonymity set is too small. Alternatively, consider submitting deposit epochs only when the anonymity set is sufficiently large.

Update: Acknowledged, not resolved. The Sunnyside Labs team stated:

This is a known issue inherent to shielded pool designs. Since we adopted N-12, which disallows zero-valued deposits, the suggested de-anonymization becomes more difficult to achieve. We will leave the design unchanged for now.

L-08 Non-Canonical Encoding of `digestRootIndices`

In `submitEpoch`, the length check for `digestRootIndices` enforces only a minimum bound (`length >= ceil(nTransfers / 64)`). However, `computeTransferDigests` consumes indices only for active transfers (`t < nTransfers`). The loop iterates across all circuit slots (`transferCount`), but digest-root decoding is only meaningful for the active portion of the batch.

As a result, any trailing words beyond `ceil(nTransfers / 64)` are never read and can be modified without affecting execution. This allows multiple calldata encodings that produce identical protocol behavior. In particular, `nTransfers` represents the number of active transfers while `transferCount` corresponds to the total circuit slot count (active transfers plus padding). Since only active transfers consume root indices, additional trailing words remain inert. While this does not introduce a security issue, it permits non-canonical calldata representations for equivalent state transitions. Off-chain systems that sign, hash, or deduplicate raw calldata may treat semantically identical requests as distinct.

Consider enforcing canonical encoding by requiring: `digestRootIndices.length == (uint256(nTransfers) + 63) / 64`.

Update: Resolved in [pull request #253](#) at commits [be26e7f](#), [c6b5e53](#), [9af5045](#), and [c305bd5](#).

L-09 Mutable `authSnapshotInterval` Can Skew `currentRound` and Break Proof Validity

`setAuthSnapshotInterval` changes [a live denominator](#) used by auth validation (`currentRound = block.number / authSnapshotInterval`), so round semantics shift instantly while proofs and snapshots were generated under prior assumptions. This can cause previously valid proofs to be rejected, or previously stale proofs to be accepted.

Consider making interval updates delayed and implementing a versioning system so that the validity of a proof depends only on the snapshot interval version at the time the proof was submitted.

Update: Resolved in [pull request #298](#) at commits [328b79e](#), [e21c161](#), and [358bb4d](#).

L-10 Forced-Withdrawal Cancellation and Execution Delays Are Equal, Making Cancellation a Race

Both `executeForcedWithdrawal` and `cancelForcedWithdrawal` call `_validateForcedWithdrawalRequest`, which unconditionally [requires](#) `block.number >= request.requestBlock + forcedWithdrawalDelay` before proceeding. This prevents preemptive cancellation during the delay and makes cancellation available only at the same moment execution becomes possible. This implies that, in the event that a user's key has been compromised, the user is in a race to cancel before their attacker executes.

Consider allowing the account owner to cancel forced-withdrawal requests during the delay window by separating the time gating for execution and cancellation. In addition, consider recording a cancellation flag that can be set during the delay and checked by `executeForcedWithdrawal`.

Update: Acknowledged, not resolved. The Sunnyside Labs team stated:

We think the suggested solutions don't actually solve the issue and perhaps introduce other issues. If user's key is compromised, there are two cases: 1) attacker wants to execute force-withdrawal to him and user tries to cancel beforehand, and 2) user has valid force-withdrawal request and attacker tries to cancel such that he can withdraw the asset to his account instead. Allowing cancellation early makes (2) more vulnerable. In addition, it makes potential attacks mentioned in M-04 easier; cancel early to invalidate TEE's proofs more frequently with same notes. Originally, the introduction of

such delay was to prevent from DoS attack to TEE. If attacker can cancel before the delay, he could cancel his forced-withdrawal and submit again.

L-11 Forced Withdrawal Requests May Lock Commitments if Execution Reverts

`requestForcedWithdrawal` stores the request and locks each input commitment in `commitmentToRequestKey` before verifying that the withdrawal parameters are executable. These locks prevent the same commitments from being used in other forced withdrawal requests. However, `withdrawal.tokenId` is not validated when the request is created. If the token is not registered (or becomes invalid before execution), `executeForcedWithdrawal` reverts in `__transferToken`. When this happens, the request and the associated commitment locks remain in place. The requester can eventually recover by calling `cancelForcedWithdrawal`, but only after the `forcedWithdrawalDelay` period enforced in `__validateForcedWithdrawalRequest`. Until then, the commitments remain locked.

Consider validating that `withdrawal.tokenId` corresponds to a registered token in `requestForcedWithdrawal` before storing the request and locking commitments. This will allow invalid requests to fail early instead of waiting for the cancellation delay.

Update: Resolved in [pull request #254](#) at commit [c8234ee](#).

L-12 `revoke` Typed Data Lacks Expiry, Allowing Delayed Relay Execution

In `AuthRegistry.sol`, the `revoke` function accepts owner-signed revocation messages submitted by allowlisted relays, but the `Revoke` EIP-712 typed data (see `REVOKE_TYPEHASH` and `__recoverRevoke`) does not include an expiry or deadline. As a result, a valid `Revoke(accountId, authPkX, nonce)` signature remains usable until the account nonce changes.

This differs from the other meta-transaction flows: `register` and `rotate` both include an expiry field and enforce it on-chain. Without a similar freshness check, a relay that obtains a valid revoke signature can delay submission and execute it later while the nonce is unchanged. This may lead to unexpected revocations if the user continues using the key after signing the message and the relay later submits the withheld signature. The scenario depends on the

account nonce remaining unchanged (i.e., neither `rotate` nor `register` being executed in the meantime).

Consider adding an `expiry` (or `deadline`) field to the `Revoke` typed data and enforcing `block.timestamp <= expiry` in `revoke`, consistent with the `register` and `rotate` flows.

Update: Resolved in [pull request #299](#) at commits [d33ce78](#), [4aaca8c](#), [4eedee0](#), and [b8ded1f](#).

L-13 Non-Canonical Commitments Accepted in Deposit Requests

The `requestDeposit()` function accepts commitments greater than or equal to `q` (the BN254 scalar field prime), while `Poseidon2T4.hash2` reduces inputs modulo `q`. As a result, `commitmentsHash` binds only to `commitment mod q`, meaning different calldata values (e.g., `x` and `x + q`) produce the same hash. The `DepositRequested` event emits the raw commitments, while `submitDepositEpoch()` later recomputes the hash from canonical field elements.

This can create inconsistencies between emitted events and the leaves appended during epoch submission. Off-chain indexers that treat `DepositRequested.commitments` as canonical leaves may desynchronize. Relay tooling that forwards non-canonical commitments into `submitDepositEpoch()` calldata may also fail verification, since the Groth16 verifier requires public inputs `< q`. Additionally, edge cases such as `commitment = q` pass the non-zero check in `requestDeposit()` but hash as `0` in-field, which the circuit rejects, leaving the request unable to be processed until cancellation.

Consider enforcing `0 < commitment < q` in `requestDeposit()`. Alternatively, consider canonicalizing commitments (`commitment mod q`) in the contract and only emitting canonical commitments in `DepositRequested`.

Update: Resolved in [pull request #255](#) at commits [f273c2f](#) and [52fc3e2](#).

Notes & Additional Information

N-01 Missing Public Input Length Check in Base Verifier

In `Groth16Verifier.sol`, the `_verifyProof` function does not validate that `publicInputs.length + 1 == vk.icLen`. Current derived verifier contracts enforce this check in their external `verify*()` functions before calling `_verifyProof()`, so the current codebase is not exploitable. In the future, if a derived contract calls `_verifyProof()` without performing this validation, the MSM may execute with mismatched input lengths, potentially causing incorrect verification behavior.

Consider enforcing the `publicInputs.length + 1 == vk.icLen` invariant directly inside `Groth16Verifier._verifyProof()`.

Update: Resolved in [pull request #256](#) at commit [768551f](#).

N-02 Unnecessary unchecked Blocks

Starting with Solidity [0.8.22](#), the compiler automatically removes overflow checks for arithmetic increments inside `for` loops. Several loops in the codebase still wrap counter increments such as `unchecked { ++i; }` in `unchecked` blocks, which is unnecessary and adds noise without providing additional gas savings.

Examples include:

- [AuthRegistry.setAllowedRelays](#)
- [AuthRegistry._updateLeaf](#)
- [PrivacyBoost.submitEpoch](#)
- [PrivacyBoost.requestForcedWithdrawal](#)
- [LibPublicInputs.buildEpochInputs](#)

Similar patterns appear throughout the codebase.

Consider removing unnecessary `unchecked` blocks around loop increments to improve code readability and maintainability.

Update: Resolved in [pull request #257](#) at commit [e351b3b](#).

N-03 Redundant Checks, Getters, Caching, and Returns

Throughout the codebase, multiple instances of redundant patterns were identified:

- **Redundant getter:** `isDepositProcessed` duplicates the autogenerated getter for the `processedDeposits` `public` state variable.
- **Unnecessary caching:** In `PrivacyBoost.submitEpoch`, `currentTreeNumber` is cached as `_currentTreeNumber` even though it is used only once.
- **Redundant zero-check:** In `_validateKnownRoots`, `root != 0` is checked before calling `isKnownTreeRoot(treeNum, root)`, but `isKnownTreeRoot` already returns `false` for `root == 0`, making the first check redundant.
- **Redundant return statements:** Some functions explicitly return values despite using named return variables (e.g., in `AuthRegistry.sol` and `Groth16Verifier.sol`).

These patterns add noise and increase code surface without providing functional benefit.

Consider removing redundant checks and wrappers, inlining single-use storage reads, and relying on implicit returns for functions with named return variables.

Update: Resolved in [pull request #258](#) at commit [6025fa8](#).

N-04 Constraint Optimizations

Some parts of the circuit introduce unnecessary constraints that do not affect correctness but increase proving cost:

- In the `appendFrontier` function of `helpers.go`, intermediate witnesses `maskedFrontier` and `maskedCarry` are `defined` to constrain `hashed` to equal `Poseidon2T4(api, frontier[i], carry)` when `merged == 1`, and `Poseidon2T4(api, 0, 0)` otherwise. However, `hashed` is only used in later computations when `merged == 1`. When `merged == 0`, the value of `hashed` is never consumed, making the requirement that it equals `zeros[1]` unnecessary. Eliminating [lines 483-484](#) and constraining `hashed := Poseidon2T4(api, frontier[i], carry)` would achieve the same behavior with fewer constraints.
- In `appendFrontier`, `appendComplete` is guaranteed to always equal `1`. The `for` `loop` iterates over the little-endian binary decomposition of `safeCount`, which is

constrained to be a `depth`-bit number. If `safeCount == maxCount - 1`, then `allBitsSet == 1` at the end of the loop; otherwise a zero bit exists and `done == 1`. In both cases `appendComplete == 1`, making the variable and the `AssertIsTrueIf(api, enabled, appendComplete)` check redundant.

- The `computeZeroHashes` function constrains `depth` Poseidon hashes on `2 * depth` intermediate witnesses. Since these values are deterministic, the `zeros` array could instead be precomputed and provided as constants within the circuit, eliminating these hashing constraints.
- In the `validateFeeTokenInputs` function of `epoch_circuit.go`, the `sanity check` is circular. The previous loop already constrains `feeActive` to be an array of `feeTokenCount` ones followed by `c.Shape.MaxFeeTokens - feeTokenCount` zeros, making the additional check redundant.

Consider removing redundant witnesses and checks, and precomputing deterministic values where possible, to reduce circuit constraints and improve proving efficiency.

Update: Resolved in [pull request #259](#) at commit [e76b070](#).

N-05 Misleading Comments

Throughout the codebase, multiple instances of misleading comments were identified:

- Inputting zero to the Poseidon2T4 hash does not return 0, as stated [here](#). Instead, it returns the following:

```
11250791130336988991462250958918728798886439319225016858543557054782819955502
```

- The comments above the `depositInternalState struct` and the `epochInternalState struct` state that they are "NOT a witness structure". This is somewhat misleading since both consist of `frontend.Variable`s, which are intermediate witnesses. A possibly better way of phrasing this is that it is not a structure for *input* witnesses.
- The documentation for the `appendFrontier` function [states](#) that it returns two values. However, it actually returns three (the third one being the final value of `carry`).
- The [constraint cost analysis](#) on `AssertRootWithTreeNumberIf` is technically correct but unnecessary: $O(15n)$ is still $O(n)$, and it does not make sense to consider the asymptotics of `len(knownRoots)`, since this is bounded by a small number (16 in this version of the protocol). For clarity, consider deleting this line.

Consider updating the comments to accurately reflect the current implementation.

Update: Resolved in [pull request #260](#) at commits [457b475](#) and [336832d](#).

N-06 Gas Optimizations

Throughout the codebase, multiple opportunities for gas optimization were identified:

- **Poseidon Round Constants Copied Per Call:** `_permute` loads the full 8KB `RC_BYTES` into memory on every call. This is acceptable for current batch sizes but may negatively impact gas scalability if `maxBatchSize` increases significantly.
- **Prefix Increment Optimization:** Use prefix increment (`++r`) instead of postfix increment (`r++`) in loops ([1](#), [2](#), [3](#)) to achieve marginal gas savings.
- **Modular Multiplication versus Subtraction:** The `LibBabyJubJub.isOnCurve` function represents $-x^2(\text{mod } p)$ as `mulmod(p - 1, mulmod(x, x, p), p)`, which is mathematically correct since $(p - 1) \cdot x^2 \equiv -x^2 \pmod{p}$. For readability and potential gas savings, this can be expressed using direct modular subtraction:

```
uint256 x2 = mulmod(x, x, p);
uint256 negX2 = x2 == 0 ? 0 : p - x2;
```

Consider assessing the trade-off between gas optimization and readability, prioritizing clarity to reduce future errors and improve transparency.

Update: Resolved in [pull request #261](#) at commits [b4543d1](#) and [f6e7d21](#).

N-07 Unused `usedRoots` Entries Permit Non-Canonical Public Inputs

The epoch flow accepts `usedRoots` entries that are known and non-zero, and only requires the inclusion of the active tree. However, [no constraint](#) enforces that every provided `(treeNumber, root)` pair is [actually consumed](#) by active inputs or digest selection. As a result, unused roots may appear in the public input set, creating non-canonical representations for equivalent state transitions. While this does not enable unauthorized spends because Groth16 binds the public inputs to the proof, it allows multiple valid proofs for identical spend semantics with different unused root supersets.

Consider enforcing canonical coverage by requiring each non-zero `usedRoots` entry to be referenced at least once by an active transfer input pair or `digestRootIndices`.

Update: Resolved in [pull request #278](#) at commits [194fdb6](#), [b697d91](#), [2f552db](#), [857fbf7](#), [306749e](#), and [8de3990](#).

N-08 OutputNPK–Ciphertext Mismatch Can Produce Unspendable Notes

[Output commitments](#) use sender-provided `OutputNPK` without verifying that it matches the key material encoded in the ciphertext. A sender can therefore construct a commitment using one `OutputNPK` while encrypting note data that derives a different key. The same commitment pattern appears in the `deposit_epoch_circuit`, where `processCommitments()` computes commitments as `Poseidon2T4(domainNote, OutputNPK, tokenId, value)`.

If such a mismatch occurs, the recipient cannot reconstruct the correct note parameters during spending, causing commitment reconstruction or nullifier derivation to fail. The resulting note becomes permanently unspendable. The sender would lose their own input funds, limiting economic incentive. However, buggy client implementations could accidentally create unspendable outputs.

Consider documenting this as a client-side correctness requirement, or introducing safeguards such as recipient-registered output keys or client-side validation checks to reduce the risk of accidental note loss.

Update: Acknowledged, not resolved. The Sunnyside Labs team stated:

| *This is a known issue we acknowledge from the design phase.*

N-09 VK Registry Key Does Not Include Full Circuit Shape

`Groth16EpochVerifier` selects verifying keys using only `(maxTransfers, maxInputsPerTransfer, maxOutputsPerTransfer)` in `verifyEpoch`. However, the Groth16 constraint system also depends on additional circuit shape parameters that are not included in this registry key. The verifier only checks `publicInputs.length + 1 == vk.icLen`, which is not a unique identifier of the compiled circuit.

For example, the epoch circuit is compiled with fixed parameters such as `NoteDepth` and `AuthDepth` (see `EpochShape`). These parameters affect the constraint system without necessarily changing the public input count. If a verifying key compiled for a circuit with

different shape parameters but identical `(T, I, 0)` values is registered, verification may fail or behave unexpectedly. This scenario requires owner misconfiguration during deployment or upgrades, as `PrivacyBoost` allows updating the verifier via `setEpochVerifier`.

Consider using a unique circuit identifier (e.g., a hash of all circuit shape parameters or a version identifier) as the verifying-key registry key, and optionally validating this identifier on-chain to prevent mismatched verifying keys during redeployments.

Update: Acknowledged, not resolved. The Sunnyside Labs team stated:

We think this brings more complication than benefits. We like hashing with circuit version in case if we have to maintain multiple versions of same circuit shape later for backward compatibility. However, the change required seems bigger as we have upgradeability.

N-10 Possible Redundant `(treeNumber, root)` Entries In Sparse Roots List

The epoch path permits `duplicate tree numbers` to support multiple roots for the same tree, but it also permits exact duplicate pairs, so the same `(treeNumber, root)` can be included multiple times without functional necessity. This follows from allowing duplicate tree numbers in epoch root validation and relying on slot-index selection rather than enforcing canonical unique pairs. This unnecessarily consumes sparse root list capacity and increases malleability of epoch proofs.

Consider preserving support for duplicate tree numbers with different roots while rejecting exact duplicate `(treeNumber, root)` pairs.

Update: Resolved in [pull request #279](#) at commit [8340788](#).

N-11 Missing and Incomplete Docstrings

Throughout the codebase, multiple instances of missing and/or incomplete docstrings were identified:

Examples of missing docstrings include:

- In `AuthRegistry.sol`: `MAX_AUTH_TREE_NUMBER`, `AUTH_ROOT_HISTORY_SIZE`, `authTreeDepth`, `authTreeRootHistory`, `allowedRelays`, and `operator`

- In `PrivacyBoost.sol`: `tokenRegistry`, `authRegistry`, `maxBatchSize`, `maxInputsPerTransfer`, `maxOutputsPerTransfer`, `maxFeeTokens`, `cancelDelay`, `forcedWithdrawalDelay`, `maxForcedInputs`, `merkleDepth`, `epochVerifier`, `depositVerifier`, and `forcedVerifier`
- In `TokenRegistry.sol`: `tokenOf`, `idOf`, and `nextId`
- In `LibDigest.sol`: `computeRequestKey`

Examples of incomplete docstrings include several digest and public-input helper functions where parameters and return values are not fully documented:

- `computeTransferDigest`, `computeWithdrawalDigest`, `computeForcedWithdrawalDigest`, `computeWithdrawalCommitment`, `computeDepositRequestId`, `computeCommitmentsHash`, `buildEpochInputs`, `buildDepositInputs`, and `buildForcedWithdrawalInputs`

Consider thoroughly documenting all functions/events (and their parameters or return values) that are part of a contract's public API. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Resolved in [pull request #280](#) at commit [792e12f](#).

N-12 Deposit Circuit Permits Zero-Value Output Commitments

The deposit flow allows commitments with `amount = 0`. While `requestDeposit` enforces a non-zero `totalAmount`, the deposit circuit only [range-checks output values](#) and does not require them to be positive. Each active commitment is appended to the note tree during batch processing (see [processCommitments](#)), so zero-value outputs still consume tree capacity. This may be intentional to support privacy padding (e.g., uniform batch sizes). In addition, relays control deposit inclusion via `submitDepositEpoch`, and deposits not included can later be cancelled via `cancelDeposit()` after the delay.

If zero-value outputs are not required for privacy padding, consider enforcing `outputValue > 0` for active outputs or documenting zero-value commitments as supported behavior.

Update: Resolved in [pull request #281](#) at commit [9f400da](#).

N-13 MSM Failure Reverts With Misleading Error

The verifier collapses multiple failure conditions during the MSM (multi-scalar multiplication) step into a single success flag and reverts with `PublicInputNotInField()` on any failure. While this error is correct when a scalar `s >= R`, it is also triggered when internal BN254 precompile calls (`ecMul` or `ecAdd`) fail.

Failures originating from precompile execution (e.g., malformed or misconfigured verifying key parameters) surface as invalid public input errors. This conflates distinct failure conditions and can hinder debugging and monitoring, since infrastructure or configuration issues become indistinguishable from user input mistakes.

Consider separating failure conditions and using distinct error selectors for scalar field violations and precompile failures. For example, revert with `PublicInputNotInField()` only when `s >= R`, and use a dedicated error (e.g., `PrecompileCallFailed()`) when `ecMul` or `ecAdd` calls fail.

Update: Resolved in [pull request #282](#) at commits [ce5d7df](#) and [720b52a](#).

Conclusion

PrivacyBoost implements a UTXO-style private asset pool where users move funds across public and shielded states through deposits, private transfers, and withdrawals, with on-chain settlement and off-chain encrypted metadata handling. The protocol's core state is maintained in append-only Poseidon Merkle trees with active-tree rollover, while spend authorization is enforced through EdDSA keys registered in a separate auth registry. Transaction batches are assembled per epoch by a TEE-operated relay, and each batch is finalized on-chain through Groth16 verification, combining privacy, double-spend protection via nullifiers, and amortized gas efficiency.

The audit identified one critical- and multiple medium-severity risks concentrated around the correctness of circuit bounds, cryptographic key validation, governance controls, and forced-withdrawal liveness. The critical issue is a tree-number range misconstraint that can prevent proving after sufficient rollovers, creating a hard protocol liveness failure. Medium-severity findings include low-order BabyJubJub key acceptance enabling signature forgery conditions, first-come account ownership assignment enabling account ID squatting, verifier-upgrade governance risk that can undermine withdrawal safety assumptions, a forced-withdrawal delay window where requested nullifiers can still be spent, requester-only forced-withdrawal execution that allows front-run griefing/DoS, and auth snapshot timing that permits revoked keys to remain usable within the current round. Lower-severity findings and notes largely relate to robustness/ canonicalization/design-hardening items, but several of these issues can still degrade operator reliability or user experience under adversarial conditions.

Overall, the architecture was found to be coherent and the intended trust boundaries clearly defined, with a strong use of zk-proof enforcement and explicit invariants in both contracts and circuits. Code quality is generally solid, but several issues demonstrate boundary mismatches between protocol intent and enforcement (especially at circuit/public-input and operational-liveness interfaces). Documentation quality is above average and materially helpful for auditability, though some comments and assumptions should be tightened to avoid ambiguity and to better align operator assumptions with enforceable on-chain guarantees.

Thanks are extended to the Sunnyside Labs team for their responsiveness throughout the audit.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.